计算机图形学
春季 25/26
2026 年 6 月 30 日

三月到六月阶段性月报告

吴与同 231220066

二维图形绘制系统月度进展报告

从核心算法原型到最终展示版本

目录

1 阶段进度简介 **3**
1.1 阶段进度总览 . . . 3

2 三月基础实现 **3**
2.1 早期代码结构 . . . 4
2.2 各个算法的具体说明 . . . 4
2.2.1 线段绘制 . . . 4
2.2.2 多边形绘制 . . . 4
2.2.3 椭圆绘制 . . . 4
2.2.4 曲线绘制 . . . 5
2.2.5 变换和裁剪 . . . 5
2.2.6 GUI 部分 . . . 5
2.3 测试和结果 . . . 6

3 三月阶段思考 **7**

4 四月增量 **7**
4.1 系统结构调整 . . . 7
4.2 GUI 交互进一步完善 . . . 8
4.2.1 交互入口优化 . . . 8
4.2.2 画布直接选中图元 . . . 8
4.2.3 从指令文件加载场景 . . . 8
4.3 四月测试补充 . . . 8
4.4 本月遇到的问题和解决方式 . . . 9
4.5 下月计划 . . . 9

5 五月增量 **10**
5.1 GUI 功能进一步完善 . . . 10
5.1.1 撤销与重做 . . . 10
5.1.2 网格和坐标提示 . . . 10
5.1.3 图元属性面板 . . . 10
5.1.4 复制、拖拽和外观编辑 . . . 11
5.1.5 快捷键和多格式导出 . . . 11
5.2 场景导出与系统结构整理 . . . 11
5.3 五月测试补充 . . . 11

5.4 本月遇到的问题和解决方式 12
5.5 下月计划 12

6 六月与最终提交前完善 13
6.1 算法对比增强 13
6.2 操作历史导出 14
6.3 批量测试可视化报告 14
6.4 最终测试统计 14
6.5 最终阶段反思 15

1 阶段进度简介

本报告汇总了三月到六月期间二维图形绘制系统的阶段性进展。整个项目的推进过程大致经历了三个层次：先把核心算法和命令行渲染流程跑通，再将 GUI 与 CLI 的场景语义统一起来，最后围绕交互、演示、复现和测试材料进行完善。回头看，这个过程并不是简单地不断添加按钮，而是逐步把“算法函数”整理成一个能够被使用、展示和验证的完整系统。

1.1 阶段进度总览

整个项目从三月开始逐步推进。三月重点是把算法和 CLI 跑通；四月重点是统一场景结构并增强 GUI 入口；五月重点是把 GUI 做成可持续使用的绘图工具；最终提交前则围绕演示、复现和测试报告补充了加分功能。表 1 概括了各阶段的主要成果。

表 1: 阶段进度总览

阶段	工作重点	主要成果
三月	核心算法和 CLI 原型	完成 DDA、Bresenham、中点椭圆、Bezier、B-spline、变换和裁剪算法，CLI 可按指令生成 BMP。
四月	GUI 入口和场景统一	增加 `cg_scene.py`，统一 CLI 与 GUI 的场景语义，支持 GUI 加载指令文件和画布点击选中。
五月	GUI 可用性完善	增加撤销重做、拖拽移动、复制、改色、图层、网格、坐标提示、属性面板和多格式导出。
最终阶段	展示、复现和测试材料	增加交互式多边形裁剪、图层管理优化、详细操作字幕反馈、算法选择弹出菜单、画布自适应网格与吸附反馈、模板场景系统、操作历史导出和可视化回归测试报告。

2 三月基础实现

三月阶段的重点是完成系统最小可用版本。也就是说，系统至少需要能够解析文本指令、调用核心算法、维护当前画布和图元状态，并在指定位置输出图像。这个阶段完成的内容如下：

- **`cg_algorithms.py` 算法部分**：
  - 线段绘制：DDA、Bresenham 都已经实现；
  - 多边形绘制：通过调用线段算法，将各条边依次连接起来完成绘制；
  - 椭圆绘制：采用中点椭圆生成算法；
  - 曲线绘制：实现了 Bezier 曲线和三次均匀 B-spline 曲线；
  - 基本变换：包括平移、旋转、缩放；
  - 线段裁剪：实现了 Cohen-Sutherland 和 Liang-Barsky 两种算法。
- **`cg_cli.py` 命令行程序**：
  - 能够读取指令文件；
  - 能够维护图元字典，并记录当前画笔颜色；
  - 能够将画布保存为 `bmp` 图片。
- **`cg_gui.py` 交互界面**：

  - 菜单中可以选择绘制模式和算法；
  - 支持通过鼠标绘制线段、椭圆、多边形和曲线；
  - 能够在右侧图元列表中选中对象，并进行基础编辑和裁剪；
  - 支持重置画布、更换颜色和保存画布。
- 此外，还编写了一些测试输入文件，并输出了几张结果图用于自测。

2.1 早期代码结构

早期代码主要分成三个模块：

模块	功能
`cg_algorithms.py`	放所有图元生成算法和几何变换。
`cg_cli.py`	解析指令文件、管理图元和画笔颜色、把像素画到位图上。
`cg_gui.py`	交互式绘制和编辑，管理画布、图元列表和菜单，直接复用 `cg_algorithms.py`。

2.2 各个算法的具体说明

2.2.1 线段绘制

线段部分首先实现了 DDA 和 Bresenham 两种算法。DDA 的思路较直观：根据横纵方向中变化较大的方向确定步数，再逐步累加浮点增量并取整。它实现简单，适合作为早期版本的正确性参照。Bresenham 则用整数误差项决定下一步像素位置，避免每步浮点运算，对任意方向和斜率的线段都能保持较稳定的离散效果。

DDA [1] 的基本思想是对直线进行增量采样。设线段两个端点分别为 (x_0, y_0) 和 (x_1, y_1)，记 $dx = x_1 - x_0$, $dy = y_1 - y_0$。算法取 $steps = \max(|dx|, |dy|)$, 然后令 $\Delta x = dx/steps$, $\Delta y = dy/steps$。从起点开始，每一步更新 $x \leftarrow x + \Delta x$，$y \leftarrow y + \Delta y$，并将当前位置取整后映射到像素网格上。这样做的好处是实现简单、思路直观，也便于检查程序输出是否正确；不足之处是算法中需要浮点加法和取整操作，累计误差也相对更明显一些 [1]。

相比之下，Bresenham 算法 [2] 采用整数增量和判别变量来决定下一个像素位置，是一种更适合光栅设备的高效画线方法 [2]。以 $0 \le k \le 1$ 的情形为例，算法沿 x 方向逐步推进，每次只需在两个候选像素之间做选择。设判别变量为 p，则可根据 p 的符号判断当前步是选择正东方向像素还是东北方向像素，并同步更新判别变量。由于整个过程只涉及整数加减，不需要乘法和除法，因此效率更高，也更适合作为正式绘制时的主要实现 [2]。

2.2.2 多边形绘制

多边形边界没有重新设计单独的扫描转换算法，而是将输入顶点按顺序连接，并对每条边复用线段生成算法。这样做的优点是实现简洁，也能自然支持 DDA 和 Bresenham 两种边界绘制方式。

2.2.3 椭圆绘制

用的中点椭圆生成算法。利用椭圆的四对称性，把绘制过程拆成两个区域，靠判别式来决定下一个像素往哪个方向走。这样避免了大量浮点运算，画出来的边界分布也比较均匀。

中点椭圆生成算法 [3, 1]。该算法利用椭圆关于坐标轴的四对称性，因此在实现时只需要计算第一象限上的点，再将得到的像素点对称映射到其余三个象限即可。这样可以明显减少重复计算。

具体来说，设椭圆中心为 (x_c, y_c)，长短半轴分别为 r_x, r_y，则标准方程为 $\frac{x^2}{r_x^2} + \frac{y^2}{r_y^2} = 1$。中点椭圆算法将第一象限的绘制过程分成两个区域：在区域 1 中，曲线斜率绝对值小于 1，因此以 x 为主变量逐步递增；在区域 2 中，曲线斜率绝对值大于等于 1，因此以 y 为主变量逐步递减 [1]。在每一步中，算法都通过判别参数判断下一个像素应当选择相邻的哪个候选点，从而逼近真实椭圆边界。

2.2.4 曲线绘制

Bezier 曲线采用 de Casteljau 递推算法进行求点 [4]。设控制点为 $P_0, P_1, \ldots, P_n$，则对参数 $t \in [0,1]$，先令 $P_i^{(0)} = P_i$，再递推计算 $P_i^{(r)}(t) = (1-t)P_i^{(r-1)}(t) + tP_{i+1}^{(r-1)}(t)$，其中 $r = 1, 2, \ldots, n$。最终得到的 $P_0^{(n)}(t)$ 就是曲线在参数 t 处的点。该方法的好处是几何意义直观，而且数值稳定性较好 [4]。

B-spline 曲线部分采用三次均匀 B 样条基函数进行采样 [5]。设控制点为 $P_0, P_1, \ldots, P_m$，则曲线可写为 $C(t) = \sum_i N_{i,3}(t) P_i$，其中 $N_{i,3}(t)$ 为三次 B-spline 基函数。与 Bezier 曲线相比，B-spline 具有局部控制的特点，即某一段曲线只受相邻少量控制点影响，因此在调整局部形状时更加方便 [5]。

在实际光栅化显示时，如果参数采样过稀，相邻采样点之间可能会出现空隙，导致曲线看起来不连续。因此在实现中，我先按参数步长采样得到离散点，再在相邻采样点之间使用 Bresenham 算法 [2] 补线。这样可以让最终显示出来的曲线更加连续、平滑。

2.2.5 变换和裁剪

平移：对所有参数点加上同一个平移向量。

旋转：围绕指定中心旋转，坐标系按题目要求使用左上角原点。

缩放：围绕指定中心按比例缩放。

裁剪：实现了 Cohen-Sutherland 和 Liang-Barsky 两种线段裁剪算法。

Cohen-Sutherland 算法 [6] 的思路是先给线段两个端点分别编码。编码一共 4 位，分别表示点是否位于裁剪窗口的左、右、下、上四个外侧区域。若两个端点编码按位或为 0，说明整条线段完全在窗口内，可以直接保留；若两个端点编码按位与不为 0，说明它们至少同时处在窗口同一侧外部，可以直接舍弃。其余情况说明线段与窗口边界有交点，此时选取一个在窗口外的端点，根据它的区域码与对应边界求交点，再用交点替换原端点，重复这个过程，直到线段被接受或舍弃。

Liang-Barsky 算法 [7] 则是从参数方程出发。设线段写成 $x = x_0 + u\Delta x,\ y = y_0 + u\Delta y,\ 0 \le u \le 1$，其中 $\Delta x = x_1 - x_0,\ \Delta y = y_1 - y_0$。裁剪窗口的四条边都可以转化为对参数 u 的不等式约束。算法通过逐步更新参数区间的进入值和离开值，判断线段在窗口内对应的有效参数范围。若最终得到的参数区间非空，就可以直接算出裁剪后的两个端点；若区间为空，则说明线段完全在窗口外。

这两种方法的思路不太一样：Cohen-Sutherland 更偏向区域分类和逐步求交，逻辑比较直观；Liang-Barsky 则利用参数区间一次性完成裁剪，计算效率更高。实际测试中，两种方法得到的裁剪结果是一致的。

2.2.6 GUI 部分

左边画布：实时显示所有图元。

右边图元列表：显示当前画布里的图元编号，点击后可以选中。

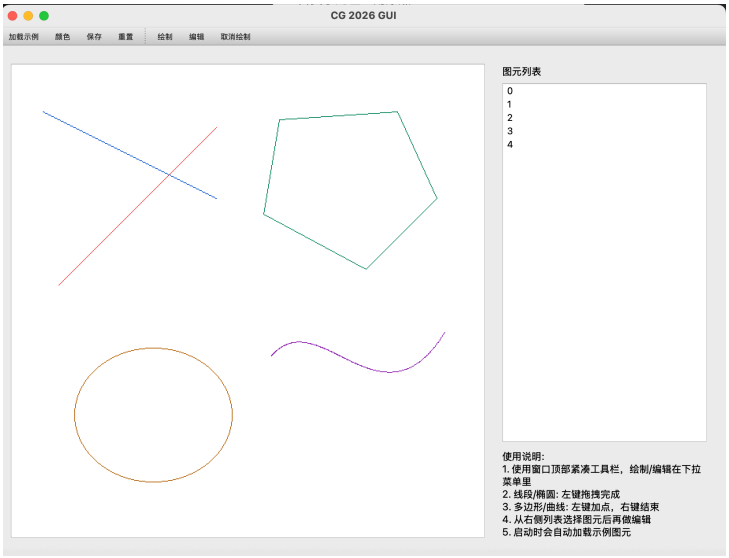


图 1: GUI 界面

顶部工具栏和菜单：用于选择绘制模式、设置颜色、执行编辑和保存画布。

交互逻辑： 选好绘制模式后，在画布上用鼠标操作。

线段和椭圆采用拖拽式输入，按下确定起点，松开确定终点。

多边形和曲线采用逐点输入，左键添加点，右键结束。

编辑时先在右侧列表中选中图元，再执行对应操作。

早期交互细节： 启动时自动加载几个示例图元，避免打开程序后面对空白画布不知从何开始。

选中图元时显示外接矩形，方便确认当前正在编辑的对象。

三月版本的 GUI 还比较基础，但已经证明了“算法模块可以被界面交互稳定调用”。后续工作的重点也因此变得清楚：继续提升选择、参数编辑、异常提示和界面组织方式。

2.3 测试和结果

这个月写了一套测试输入，覆盖了基础图元和变换操作：

- 线段绘制测试
- 多边形绘制测试
- 平移与缩放测试
- 椭圆与线段裁剪测试
- Bezier 与 B-spline 曲线测试

三月测试输出见图示结果：

GUI 那边也做了基础验证，主要看了这些：

- 启动后示例图元能不能正常加载
- 各种绘制模式切换是否正常
- 鼠标画线段、椭圆、多边形、曲线能不能正确输入
- 选中图元后平移、旋转、缩放、裁剪、删除是否正常
- 导出位图能不能正常工作

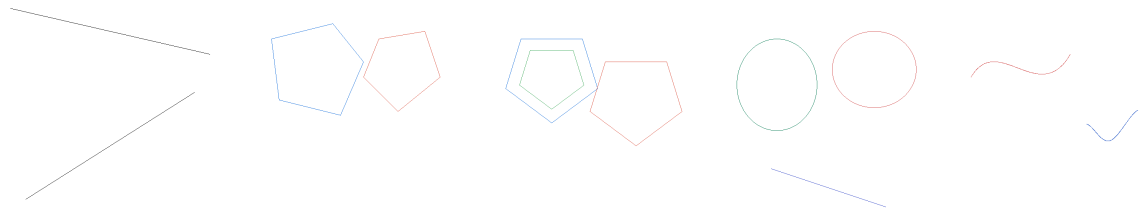


图 2: 测试结果图，从左到右分别为线段绘制、多边形绘制、平移与缩放、椭圆与裁剪、Bezier 与 B-spline 曲线。

3 三月阶段思考

曲线断裂问题：只绘制离散采样点时，曲线会出现断点。后来在相邻采样点之间使用 Bresenham 补线，才让曲线在视觉上连续。

旋转方向问题：屏幕坐标系下 y 轴向下，视觉上的旋转方向与常见数学坐标系不同。实现和测试时必须统一坐标约定，否则 CLI 输出和 GUI 显示容易出现理解偏差。

4 四月增量

在三月已经完成核心算法、CLI 基础版本和 GUI 原型的基础上，四月的工作重点主要放在两个方面：一是继续完善 GUI 的交互逻辑，使其更接近一个可以实际演示和使用的绘图系统；二是补充边界情况测试，并对代码结构进行整理，减少 CLI 与 GUI 中重复的场景解析逻辑。

与三月相比，本月新增和完善的内容主要包括：

- 进一步改进 GUI 的交互方式，包括启动时自动加载示例图元、整理顶部工具栏、支持画布直接选中图元，以及支持从指令文件中加载场景；
- 新增共享场景解析模块 `cg_scene.py`，用于统一指令文件解析、图元状态维护和几何编辑操作；
- 补充四月测试输入，覆盖特殊线段、反向包围框椭圆、裁剪边界情况和曲线测试；
- 生成新的测试结果图，用于报告展示和后续回归检查。

4.1 系统结构调整

截至四月底，系统主要由以下几个模块组成：

模块	功能
`cg_algorithms.py`	实现线段、多边形、椭圆、曲线、变换和裁剪等核心算法
`cg_scene.py`	统一解析指令文件、维护场景状态、复用图元数据结构
`cg_cli.py`	根据指令文件顺序执行绘制、编辑和保存操作，输出位图结果
`cg_gui.py`	提供交互式绘制和编辑界面，直接复用算法和场景解析模块

表 2: 四月底系统模块结构

本月增加 `cg_scene.py` 的主要原因是：三月版本中，CLI 和 GUI 都需要理解同一套图元数据和指令语义。如果这两部分分别维护，后续功能增多时容易出现逻辑不一致的问题。因此四月对代码结构做了一次整理，把命令解析、图元构造、平移、旋转、缩放、裁剪等操作统一收口到共享场景模块中。经过这一轮整理后，CLI 和 GUI 都可以复用同一层逻辑，后续继续扩展时会更加稳妥。

4.2 GUI 交互进一步完善

三月的 GUI 原型已经能够绘制和编辑图元，但整体入口还比较分散，而且在 macOS 上过于依赖系统顶部菜单栏，刚打开时容易误以为界面是空的。四月对 GUI 的交互入口和对象选择方式进行了进一步完善。

4.2.1 交互入口优化

本月对 GUI 的顶部操作区做了重新整理。常用操作被保留在窗口内工具栏中，例如加载示例、加载指令、设置颜色、保存和重置。绘制和编辑功能则被收纳到下拉式工具按钮中，使顶部界面更加紧凑。

这样调整之后，GUI 更适合课堂展示和录屏演示。用户打开程序后，可以直接从窗口内部看到主要功能入口，不需要过度依赖系统菜单栏。同时，右侧图元列表仍然保留，作为显式的对象管理入口。

4.2.2 画布直接选中图元

三月版本中，对象选择主要依赖右侧图元列表。本月新增了在画布中直接点击图元进行选择的功能。

为了实现这一点，我在图元对象中补充了更贴近像素绘制结果的命中判断逻辑，而不是只依赖外接矩形。这样做之后，线段、椭圆、曲线等图元在画布上的选中体验更自然。例如，对于一条很细的线段，如果只用外接矩形判断，可能会出现点击空白区域也被选中的情况；改成基于实际绘制像素附近的命中判断后，选中结果会更符合用户直觉。

这一功能也为后续继续实现拖拽编辑、直接移动图元等交互打下了基础。

4.2.3 从指令文件加载场景

为了让 GUI 与 CLI 之间形成更好的联动，本月增加了从文本指令文件加载场景的功能。这样一来，CLI 中已经验证过的输入文件可以直接在 GUI 中打开，既方便复现实验结果，也方便在图形界面中展示测试场景。

这一功能也体现了新增 `cg_scene.py` 的价值。GUI 不再需要自己重新解释一套命令语义，而是直接调用共享场景解析逻辑。这样可以减少重复代码，也能保证 CLI 和 GUI 对同一份输入文件的理解是一致的。

4.3 四月测试补充

在三月已有测试图的基础上，本月新增了两个测试输入文件：

- `input_april.txt`：用于覆盖更多边界情况和典型情形；
- `input_april_gui_demo.txt`：用于验证 GUI 从指令文件加载场景的能力。

对应生成的输出图包括：

四月新增测试输出见图示结果：

GUI 部分本月重点检查了以下交互流程：

- 启动后是否能够正确显示示例图元；
- 是否能够从工具栏下拉菜单进入绘制和编辑模式；
- 是否能够直接在画布中点击选中图元；
- 是否能够从指令文件加载已有测试场景；

文件名	主要验证内容
`april_1_lines.bmp`	水平线、竖直线、负斜率线段、退化为单点的线段
`april_2_shapes.bmp`	多边形、旋转、椭圆以及对角顶点顺序变化的情况
`april_3_clip.bmp`	裁剪窗口内、穿过窗口边界、完全在窗口外的线段
`april_4_curves.bmp`	Bezier 与三次均匀 B-spline 曲线

表 3: 四月新增测试输出

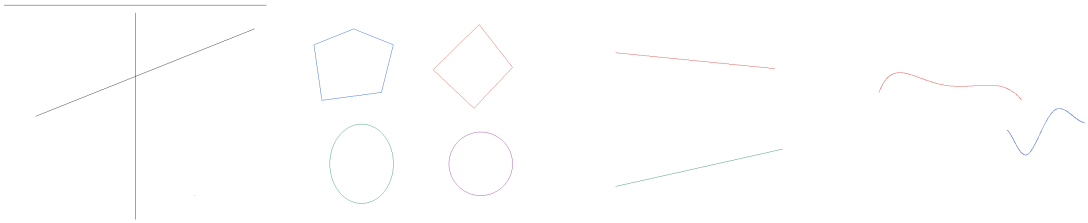


图 3: 四月测试结果图，从左到右分别为线段边界情况、图形与变换、裁剪边界情况、Bezier 与 B-spline 曲线。

- 保存位图时是否会正确隐藏预览图元和选中框。

整体来看，四月版本的 GUI 已经比三月版本更接近一个完整的课程项目原型。它不仅能够手动绘制和编辑图元，也能够读取已有指令文件进行展示，因此 CLI 和 GUI 之间的关系更加清晰。

4.4 本月遇到的问题和解决方式

四月继续遇到并处理了以下几个问题。

GUI 入口不够直观。早期版本在 macOS 上主要依赖系统顶部菜单栏，刚打开时容易误以为界面是空的。本月通过窗口内工具栏和示例图元加载缓解了这个问题，使用户打开程序后能更快理解当前系统可以做什么。

CLI 与 GUI 解析逻辑重复。如果两个界面分别维护一套场景解析流程，后续修改指令支持时很容易出现不一致。因此本月新增了共享场景模块 `cg_scene.py`，将解析逻辑统一。这样既减少了重复代码，也方便后续维护。

画布选中判定不自然。仅依赖图元外接矩形时，细线段和曲线的选中体验不够理想，可能会出现点击图元附近空白区域却误选中的情况。因此本月改为基于实际绘制像素附近的命中判断，提高了对象选择的准确性和可用性。

4.5 下月计划

五月计划继续推进以下内容：

- 继续完善 GUI 交互，例如更细致的参数输入方式和更自然的对象编辑流程；
- 补充更多极端情况测试，并整理更系统的测试结果；
- 进一步打磨界面表现和使用体验，为最终演示视频做准备；
- 开始整理最终系统报告和说明书需要的素材，包括截图、功能说明和测试对比图。

5 五月增量

在三月和四月已经完成核心算法、CLI 基础流程、共享场景模块和 GUI 交互原型的基础上，五月的工作重点主要放在进一步完善系统的可用性和展示效果上。本月没有继续新增新的核心图形学算法类别，而是围绕 GUI 的交互体验、操作恢复、场景导入导出和测试材料进行补充，使系统更接近最终展示版本。

与四月相比，本月新增和完善的内容主要包括：

- 进一步增强 GUI 功能，包括撤销、重做、复制图元、拖拽移动图元、选中图元改色以及图元层级调整；
- 增加画布网格显示、鼠标坐标提示和图元属性面板，使绘制和调试过程更加直观；
- 补充常用快捷键，例如撤销、重做、复制和删除，减少重复操作时的负担；
- 在四月 `cg_scene.py` 的基础上增加场景导出能力，使 GUI 当前场景可以保存为符合 CLI 格式的指令文件；
- 保存画布时除了 `bmp`，也支持导出为 `png` 和 `jpg`，方便后续报告排版和演示材料整理；
- 新增五月测试输入 `input_may.txt`，并生成多张结果图，用于展示综合场景、变换对比、裁剪对比和曲线椭圆效果。

5.1 GUI 功能进一步完善

四月版本的 GUI 已经能够通过工具栏进入绘制和编辑流程，也支持从指令文件加载场景。五月主要继续补强 GUI 在连续使用时的体验，使它更像一个可以实际操作的绘图程序。

5.1.1 撤销与重做

之前版本在执行平移、旋转、缩放、裁剪、删除、加载场景等操作时，如果操作失误，就只能重新绘制或重新加载。本月给 GUI 增加了撤销和重做功能。每次执行会改变场景状态的操作前，程序都会保存当前场景快照；撤销时恢复上一个快照，重做时再恢复被撤销的快照。

这个功能对实际使用比较重要。比如在演示过程中，如果输入的旋转角度不合适，或者误删了某个图元，就可以直接撤销，而不需要重新构造整个场景。它也让 GUI 的容错性更好。

5.1.2 网格和坐标提示

为了让绘制过程更直观，本月在画布中增加了可开关的网格显示，并在状态栏实时显示鼠标所在坐标。这样在绘制线段、椭圆、多边形和曲线时，可以更容易判断当前点的位置，也方便检查坐标系是否符合题目要求。

这个功能本身不涉及新的图形学算法，但对调试很有帮助。尤其是在检查线段端点、椭圆外接矩形、多边形顶点和曲线控制点时，坐标提示可以帮助我更快判断输入是否正确。

5.1.3 图元属性面板

本月在右侧面板中增加了图元属性显示。选中图元后，界面会显示该图元的编号、类型、使用算法、当前颜色、参数点数量以及原始参数点列表。

三月和四月版本中，右侧列表主要只能看出当前有哪些图元，但不能直接看到它们的具体参数。加入属性面板后，GUI 中显示的信息和底层图元数据之间的对应关系更加清楚。比如选中一条线段后，可以直接看到它的两个端点；选中曲线后，也可以看到控制点数量和具体坐标。这对调试和最终展示都比较方便。

5.1.4 复制、拖拽和外观编辑

为了让交互编辑更自然，本月增加了复制图元和鼠标拖拽移动图元的功能。复制图元可以快速构造对比图形，比如先复制一个多边形，再对其中一个进行平移、旋转或缩放，用来观察变换前后的差异。拖拽移动则让用户不必每次都通过输入参数来移动对象。

拖拽功能实现时还需要注意撤销记录的问题。如果鼠标每移动一个像素就记录一次撤销，撤销栈会变得很混乱。因此我在实现中只在一次拖动开始产生位移时记录一次状态，之后拖动过程持续更新图元坐标。这样撤销时可以回到拖动前的状态，而不是一步一步回退每个像素。

此外，本月还增加了选中图元改色、上移一层和下移一层的功能。改色可以让展示效果更清楚；层级调整可以处理多个图元重叠时的遮挡关系。虽然这些功能不是课程基础算法要求中的重点，但能明显提升系统作为绘图工具的完整度。

5.1.5 快捷键和多格式导出

本月还补充了一些常用快捷键：

- `Ctrl+Z`：撤销；
- `Ctrl+Shift+Z`：重做；
- `Ctrl+D`：复制当前图元；
- `Delete`：删除当前图元。

这些快捷键主要是为了减少重复点击菜单的操作。对于最终演示来说，快捷键也能让操作过程更顺畅。

保存画布方面，四月之前主要输出 `bmp` 文件。本月在 GUI 中增加了 `png` 和 `jpg` 等格式的导出支持。这样报告中如果需要插图，可以直接使用更常见的图片格式，不一定每次都只依赖 `bmp`。

5.2 场景导出与系统结构整理

四月新增 `cg_scene.py` 后，CLI 和 GUI 已经可以复用同一套场景解析逻辑，GUI 也可以直接加载 CLI 使用的指令文件。五月在这个基础上继续补充了反向导出功能，也就是把 GUI 当前场景保存成文本指令文件。

这样一来，CLI 和 GUI 之间形成了双向联动：

- CLI 使用的输入文件可以加载进 GUI，用于交互式查看和编辑；
- GUI 中编辑后的场景也可以导出为指令文件，再交给 CLI 复现输出结果。

这个功能对最终报告和演示都比较有帮助。因为 GUI 中手动构造的场景不再只能停留在界面里，而是可以保存成文本指令，之后继续复现、修改或作为测试输入使用。同时，这也进一步说明 `cg_scene.py` 的作用不是单纯减少重复代码，而是让系统中的场景数据有了更统一的表示方式。

5.3 五月测试补充

本月新增测试输入文件 `input_may.txt`。相比三月和四月的测试，五月测试更偏向综合展示，不只是检查单个算法是否能运行，而是检查多个图元和编辑操作组合后是否仍然能够稳定输出。

本月生成的测试输出包括：

五月测试输出见图示结果：

GUI 部分本月重点检查了以下交互流程：

- 撤销和重做是否能够正确恢复场景状态；
- 图元复制、删除、改色、拖拽移动后，画布和图元列表是否同步更新；

文件名	主要验证内容
`may_1_scene.bmp`	综合场景绘制，包括线段、多边形、椭圆和 Bezier 曲线
`may_2_transform_compare.bmp`	多边形旋转、缩放和平移的对比效果
`may_3_clip_compare.bmp`	Cohen-Sutherland 与 Liang-Barsky 裁剪结果对比
`may_4_curves_ellipse.bmp`	高阶 Bezier、B-spline 以及反向包围框椭圆

表 4: 五月新增测试输出

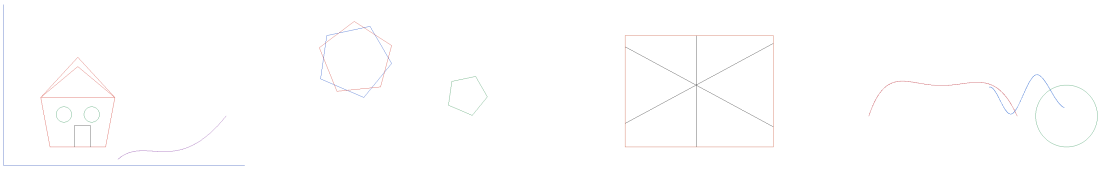


图 4: 五月测试结果图，从左到右分别为综合场景、变换对比、裁剪对比、曲线与椭圆测试。

- 网格显示和鼠标坐标提示是否能辅助定位；
- 属性面板中显示的图元类型、算法、颜色和参数是否与实际数据一致；
- GUI 当前场景是否能够正确导出为指令文件；
- 保存画布时，`bmp`、`png` 和 `jpg` 格式是否能够正常输出。

五月系统能够把算法、场景管理、GUI 操作和文件导出连接到一起。CLI 更适合批量生成测试图，GUI 则更适合交互式构造和展示场景。

5.4 本月遇到的问题和解决方式

五月主要遇到并处理了以下几个问题。

撤销恢复的状态粒度问题。撤销功能不能只保存某一个图元，否则加载场景、删除图元、重置画布等操作很难正确恢复。因此本月采用保存整个场景快照的方式。这样虽然会多占用一些内存，但逻辑更简单，也更稳定。

GUI 与 CLI 的双向一致性问题。四月版本已经支持 GUI 加载指令文件，但如果 GUI 编辑后的结果不能重新导出，就会导致 GUI 和 CLI 之间仍然没有完全打通。本月增加指令文件导出后，GUI 编辑结果也可以被 CLI 复现，两个入口之间的一致性更好。

界面信息不足的问题。只有画布和图元列表时，选中对象的具体参数不够直观。本月加入图元属性面板后，可以直接看到算法、颜色和控制点数据，调试和展示都更方便。

拖拽编辑的撤销记录问题。如果拖动图元时每移动一个像素都记录一次撤销，撤销栈会变得很混乱。因此实现时只在一次拖动开始产生位移时记录一次状态，拖动过程持续更新图元坐标。这样既保留了撤销功能，也避免了撤销记录过细。

5.5 下月计划

六月到最终提交前，主要工作将从继续增加功能转向整理和交付。后续计划主要包括

- 整理最终系统报告，把各类算法原理、实现细节和测试结果统一组织起来；
- 完成系统使用说明书，补充 GUI 操作截图和功能说明；
- 录制系统演示视频，展示 CLI 与 GUI 的主要功能；
- 对代码进行最后一轮清理，确保最终压缩包结构符合课程要求。

6 六月与最终提交前完善

在五月版本已经具备完整 GUI 编辑能力的基础上，最终阶段的工作重点从“继续增加基础操作”转向“让系统更适合展示、复现和验收”。因此本阶段主要补充了以下功能：交互式多边形裁剪、图层管理与遮挡优化、详细操作字幕反馈系统、算法选择弹出菜单、画布自适应网格与吸附反馈、算法对比增强、操作历史导出和批量测试可视化报告。这些功能不改变 README 规定的 CLI 指令格式，而是围绕已有图元和算法做更清晰的展示和更流畅的交互体验。

本阶段新增和完善的内容主要包括：

- 实现交互式多边形裁剪，选中多边形后拖拽裁剪窗口实时预览 Sutherland-Hodgman 结果；
- 优化图层遮挡管理，修正 Z 序方向并改用实心画刷渲染填充多边形；
- 增加详细操作字幕系统，覆盖绘制、选择、编辑、变换、裁剪、图层、文件、网格吸附和演示窗口等操作；字幕支持较长文本自动换行，用于说明算法含义、操作结果和边界情况；
- 左侧工具箱线段和多边形按钮增加算法选择弹出菜单；
- 为命令编辑器补充错误行高亮和错误保护，非法指令不会覆盖原有画布；
- 扩展算法对比窗口，增加像素数量、运行耗时和算法差异说明；
- 增加裁剪算法对比，对同一条线段同时展示 Cohen-Sutherland 与 Liang-Barsky 的裁剪结果；
- 增加操作历史导出功能，将 GUI 操作日志和当前可复现 CLI 指令一起保存；
- 增强 `run_regression_tests.py`，生成 `report.html` 和 `report.md`，展示输出图、非白像素数量和算法覆盖情况。

6.1 算法对比增强

本阶段继续扩展了原来的算法对比窗口。最初的对比更偏向“并排显示结果”，最终版本则增加了统计信息，使窗口更能体现图形学课程内容。

当前算法对比主要支持以下几组：

表 5: 最终阶段算法对比内容

图元或操作	对比算法	显示信息
线段	Bresenham 与 Wu 反走样	像素点数量、运行耗时、整数线段与反走样线段的视觉差异。
多边形	DDA 与 Bresenham	每条边复用线段算法，统计整体边界像素。
曲线	Bezier 与 B-spline	控制点数量、是否经过首末控制点、曲线像素数量。
线段裁剪	Cohen-Sutherland 与 Liang-Barsky	裁剪窗口、裁剪后端点、是否被拒绝、运行耗时。

这个功能的意义在于，它不只是为了界面好看，而是把报告中需要解释的算法差异直接放到了系统里。例如 DDA 和 Bresenham 的输出像素可能接近，但计算方式不同；Bezier 曲线通常经过首末控制点，而 B-spline 更强调平滑和局部控制；两种裁剪算法得到的几何结果应当一致，但推导思路分别来自区域码和参数区间。

6.2 操作历史导出

GUI 中很多操作是交互式完成的，如果只保存最终图片，无法看出这个场景是怎样构造出来的。最终阶段增加了“导出操作历史”功能，用来保存更完整的实验记录。

导出的文本文件包含以下内容：

- 导出时间和当前画布尺寸；
- 当前场景对应的可复现 CLI 指令；
- GUI 操作日志，例如绘制、加载模板、平移、旋转、缩放、裁剪、复制、删除和导出等记录。

这个功能和“导出指令文件”有所区别。导出指令文件只关注最终场景如何被 CLI 复现；操作历史导出则更适合写报告和保留演示过程，因为它能说明用户在 GUI 中实际做过哪些操作。这样一来，GUI 操作也具有一定的可追溯性。

6.3 批量测试可视化报告

最终阶段还增强了批量测试脚本。原来的回归测试主要负责运行输入文件并确认输出 BMP 非空；增强后，带 `--report-dir` 参数运行时会生成固定的 HTML 和 Markdown 报告。

运行方式如下：

```
python3 source/run_regression_tests.py --output-dir /tmp/cg_outputs
python3 source/run_regression_tests.py --report-dir /tmp/cg_report
```

Listing 1: 批量测试与报告生成命令

报告内容包括每个输入文件生成的图片、图片尺寸、非白像素数量、涉及的命令类型和算法覆盖情况。算法覆盖汇总中会列出 DDA、Bresenham、Bezier、B-spline、Ellipse、Cohen-Sutherland 和 Liang-Barsky 是否被覆盖。

这个功能对最终提交很有帮助。它让测试结果不只停留在终端输出里，而是变成可以直接打开查看的 `report.html`。在修改 GUI 或场景解析逻辑之后，也可以快速运行脚本确认 CLI 输出是否仍然正常。

6.4 最终测试统计

最终版本继续使用三月、四月和五月输入文件进行回归测试。带 `saveCanvas` 的 3 个输入文件共生成 13 张非空 BMP 图像，另一个 GUI 演示输入文件因为没有保存指令而被跳过。主要统计如表 6 所示。

表 6: 最终回归测试统计

输入文件	输出图数	非白像素数量	覆盖重点
`input_march.txt`	5	962, 1260, 1779, 1533, 557	基础线段、多边形、椭圆、曲线、变换和裁剪。
`input_april.txt`	4	1733, 2004, 822, 812	特殊线段、退化线段、反向椭圆包围框和裁剪边界情况。
`input_may.txt`	4	3372, 1645, 2994, 1716	综合场景、变换对比、裁剪对比、高阶曲线和椭圆。
合计	13	全部非空	覆盖课程要求的主要图元生成、几何变换和线段裁剪算法。

从最终测试结果看，系统能够稳定处理普通输入和若干边界情况。尤其是退化线段、反向输入椭圆包围框、完全在裁剪窗口外的线段等情况都没有导致程序崩溃或输出空图。

6.5 最终阶段反思

最终阶段让我更明显地感受到，课程项目后期的重点不只是“功能有没有”，还包括“能不能稳定展示、能不能复现、能不能说明清楚”。交互式裁剪和图层遮挡优化解决的是交互完整性，详细操作字幕系统解决的是操作可感知性和录屏说明问题，算法对比窗口解决的是课程内容可视化问题，操作历史导出解决的是 GUI 操作可追溯问题，批量测试报告解决的是验收和回归问题。

这些功能都没有改变底层核心算法，但它们把算法、界面、文件和测试连接得更完整。对最终提交而言，这比单纯继续堆叠零散按钮更有价值，也更容易在报告和演示视频中讲清楚系统的完整性。

参考文献

[1] John F Hughes. *Computer graphics: principles and practice*. Pearson education, 2014.

[2] Jack E Bresenham. Algorithm for computer control of a digital plotter. In *Seminal graphics: pioneering efforts that shaped the field*, pages 1–6. 1998.

[3] Jerry R Van Aken. An efficient ellipse-drawing algorithm. *IEEE Computer Graphics and Applications*, 4(9):24–35, 2007.

[4] Gerald E Farin. *Curves and surfaces for CAGD: a practical guide*. Morgan Kaufmann, 2002.

[5] Les Piegl and Wayne Tiller. Rational b-spline curves and surfaces. In *The NURBS book*, pages 117–139. Springer, 1997.

[6] William M Newman and Robert F Sproull. *Principles of interactive computer graphics*. McGraw-Hill, Inc., 1979.

[7] You-Dong Liang and Brian A. Barsky. A new concept and method for line clipping. *ACM Transactions on Graphics (TOG)*, 3(1):1–22, 1984.